VPN ID Quoting Fix — Test Procedure

Branch: vpn-id-quoting-fix · Commit: 29b0a492

Privoro / Akita VPN team

2026-05-08

Overview

This document describes the test procedure for verifying the changes introduced in the `vpn-id-quoting-fix` branch.

Scope of changes:

- `pkcs11-meta.conf` values are single-quoted so sourcing is safe with parens / spaces (e.g. `'CN=Foo (Bar)'`).
- `LOCAL_ID` / `REMOTE_ID` are scoped to PSK only; PUBKEY mode clears them.
- SCLI rejects identifiers and secrets that the IPsec parsers cannot safely express (whitespace in non-DN IDs, `"` or `\` in ASCII secrets).
- DN values are quoted in `swanctl.conf`, `swanctl.secrets.psk`, `ipsec.conf`, `ipsec.secrets`. `0x` / `0s` secrets stay unquoted for binary decode.

Target: STM32MP15 (Akita platform, shiba-meta-secure-boot).

0. Prerequisites

0.1 Build and deploy

Two deployment options.

Option A — Full SWU (recommended for end-to-end test):

```
~/bin/merge-epl-next.sh                    # rebuild epl-next +
        push

cd <yocto build dir>
bitbake swu-image-secure

scp tmp/deploy/images/.../*.swu root@<target>:/tmp/
ssh root@<target> '/usr/sbin/update-sw.sh /tmp/*.swu && systemctl
        reboot'
```

Option B — Quick partial deploy (CLI + shell scripts only):

```
GOOS=linux GOARCH=arm GOARM=7 CGO_ENABLED=0 go build \
  -C recipes-admin/cmd-line-interface/files/scli -o /tmp/scli-
        armhf .

TARGET=root@<target-ip>
scp /tmp/scli-armhf "$TARGET:/usr/bin/scli"
scp recipes-security/akita-pki/files/bin/pkcs11-save-meta.sh
        "$TARGET:/usr/sbin/"
scp recipes-security/vpn-pkcs11-pin/files/vpn-pkcs11-pin.sh
        "$TARGET:/usr/sbin/"
ssh "$TARGET" 'chmod 755 /usr/bin/scli /usr/sbin/pkcs11-save-
        meta.sh /usr/sbin/vpn-pkcs11-pin.sh'
```

0.2 Backup current state

```
ssh $TARGET 'cp /var/lib/akita-vpn/pkcs11-meta.conf{,.bak}'
```

Test Group 1 — SCLI Input Validation

1.1 Valid IDs accepted

```
ssh $TARGET sudo scli <<'EOF'
security vpn set psk local-id 192.168.1.10
security vpn set psk local-id vpn.example.com
security vpn set psk local-id user@example.com
security vpn set psk local-id %fromcert
security vpn set psk local-id "CN=Test Server, O=Acme"
exit
EOF
```

Pass criteria: 5 commands, each prints `Staged: PSK local ID to ...`.

1.2 Invalid IDs rejected

```
ssh $TARGET sudo scli <<'EOF'
security vpn set psk local-id "branch office"
security vpn set psk local-id "site west"
security vpn set psk local-id "%fromcert bad"
security vpn set psk local-id "  "
security vpn set psk local-id ""
exit
EOF
```

Pass criteria: 5 rejections with one of:

- `Error: ID with whitespace must be a Distinguished Name ...`
- `Error: magic ID (%...) must not contain whitespace`
- `Error: ID is empty`

`cfg.LocalID` MUST remain unchanged (verify with `security vpn show`).

1.3 Canonical form (trim leading/trailing whitespace)

```
ssh $TARGET sudo scli <<'EOF'
security vpn set psk local-id "  192.168.1.10  "
security vpn show
```

```
exit
EOF
```

Pass criteria: shown LocalID is `192.168.1.10` (trimmed, no surrounding whitespace).

1.4 Secret validation

```
ssh $TARGET sudo scli <<'EOF'
security vpn set psk secret 'GoodSecret123'
security vpn set psk secret 0x3A5C0D50A4FA16A92813D2F662193323
security vpn set psk secret 0sQUJDREVGRw==
security vpn set psk secret 'Bad"Secret'
security vpn set psk secret 'Bad\Secret'
exit
EOF
```

Pass criteria: first 3 accepted (Staged), last 2 rejected with `Error: ASCII secret must not contain '"' or '\'.`

Test Group 2 — pkcs11-meta.conf

2.1 Single-quote applied

```
ssh $TARGET sudo scli <<'EOF'
security vpn set auth psk
security vpn set psk local-id 192.168.1.10
security vpn set psk remote-id "CN=CISCO VPN Server (Privoro CISCO
        ASA)"
security vpn set psk secret 'TestSecret123'
exit
EOF
ssh $TARGET 'cat /var/lib/akita-vpn/pkcs11-meta.conf'
```

Pass criteria: every value wrapped in `'...'`. Example:

```
CERT_NAME='vpn-client'
PKCS11_HANDLE='0x...'
LOCAL_ID='192.168.1.10'
REMOTE_ID='CN=CISCO VPN Server (Privoro CISCO ASA)'
```

2.2 Sourcing is parser-safe

```
ssh $TARGET 'bash -c ". /var/lib/akita-vpn/pkcs11-meta.conf && \
  echo OK: REMOTE_ID=[\$REMOTE_ID]"'
```

Pass criteria: prints `OK: REMOTE_ID=[CN=CISCO VPN Server (Privoro CISCO ASA)]`. No shell syntax error from the parens.

2.3 PSK -> PUBKEY transition clears LOCAL_ID / REMOTE_ID

```
ssh $TARGET sudo scli <<'EOF'
security vpn set auth pubkey
security vpn set certs key-cert vpn-client
exit
EOF
ssh $TARGET 'cat /var/lib/akita-vpn/pkcs11-meta.conf'
```

Pass criteria: no `LOCAL_ID` or `REMOTE_ID` lines remain; only `CERT_NAME` and `PKCS11_HANDLE` (plus `PPK_ID` if configured) are present.

2.4 PUBKEY -> PSK transition writes new LOCAL_ID / REMOTE_ID

```
ssh $TARGET sudo scli <<'EOF'
security vpn set auth psk
security vpn set psk local-id 10.0.0.1
security vpn set psk remote-id 10.0.0.2
security vpn set psk secret 'TestSecret123'
exit
EOF
ssh $TARGET 'cat /var/lib/akita-vpn/pkcs11-meta.conf'
```

Pass criteria: `LOCAL_ID='10.0.0.1'` and `REMOTE_ID='10.0.0.2'` are present.

Test Group 3 — strongSwan output

3.1 Switch engine

```
ssh $TARGET sudo scli <<'EOF'
security vpn set engine strongswan
exit
EOF
```

3.2 DN with whitespace is quoted

```
ssh $TARGET sudo scli <<'EOF'
security vpn set auth psk
security vpn set psk local-id "CN=Branch Office, O=Acme"
```

```
security vpn set psk remote-id "CN=HQ Server, O=Acme"
security vpn set psk secret 'TestSecret'
security vpn save
exit
EOF
ssh $TARGET 'sudo cat /etc/swanctl/conf.d/*.conf | grep -A3 "ike-
        psk\|local {\|remote {"'
ssh $TARGET 'sudo cat /run/vpn/swanctl.secrets.psk'
```

Pass criteria:

`swanctl.conf`:

```
local  { auth = psk; id = "CN=Branch Office, O=Acme" }
remote { auth = psk; id = "CN=HQ Server, O=Acme" }
```

`swanctl.secrets.psk`:

```
ike-psk {
    id-local = "CN=Branch Office, O=Acme"
    id-remote = "CN=HQ Server, O=Acme"
    secret = "TestSecret"
}
```

3.3 strongSwan starts cleanly

```
ssh $TARGET 'systemctl restart strongswan && \
  systemctl status strongswan --no-pager | head -15'
ssh $TARGET 'journalctl -u strongswan --since "1 minute ago" \
  | grep -E "ERROR|FATAL" | head'
```

Pass criteria: status `active (running)`. No ERROR / FATAL log lines.

Test Group 4 — libreswan output

4.1 Switch engine

```
ssh $TARGET sudo scli <<'EOF'
security vpn set engine libreswan
exit
EOF
```

4.2 DN with whitespace is quoted

```
ssh $TARGET sudo scli <<'EOF'
security vpn set auth psk
security vpn set psk local-id "CN=Branch Office, O=Acme"
```

```
security vpn set psk remote-id "CN=HQ Server, O=Acme"
security vpn set psk secret 'TestSecret'
security vpn save
exit
EOF
ssh $TARGET 'grep -E "leftid|rightid" /etc/ipsec.conf'
ssh $TARGET 'sudo cat /run/vpn/ipsec.secrets'
```

Pass criteria:

`ipsec.conf`:

```
leftid="CN=Branch Office, O=Acme"
rightid="CN=HQ Server, O=Acme"
```

`ipsec.secrets`:

```
"CN=Branch Office, O=Acme" "CN=HQ Server, O=Acme" : PSK
"TestSecret"
```

4.3 addconn parses without error

```
ssh $TARGET 'systemctl restart ipsec && \
  systemctl status ipsec --no-pager | head -15'
ssh $TARGET 'journalctl -u ipsec --since "1 minute ago" \
  | grep -E "ERROR|FATAL|unrecognized" | head'
```

Pass criteria: status `active (running)`. No `unrecognized keyword`, `not a numeric IPv4 address`, ERROR or FATAL lines.

4.4 PSK + IP / FQDN (NIAP guaranteed range)

```
ssh $TARGET sudo scli <<'EOF'
security vpn set psk local-id 192.168.1.10
security vpn set psk remote-id vpn.example.com
security vpn save
exit
EOF
ssh $TARGET 'sudo cat /run/vpn/ipsec.secrets'
ssh $TARGET 'systemctl status ipsec --no-pager | head -5'
```

Pass criteria: `192.168.1.10 @vpn.example.com : PSK "..."` is written and `ipsec` is `active (running)`.

Test Group 5 — Secret formats

5.1 ASCII secret -> quoted in output

```
ssh $TARGET sudo scli <<'EOF'
security vpn set psk secret 'MyStrongPassphrase123!'
security vpn save
exit
EOF
ssh $TARGET 'sudo cat /run/vpn/swanctl.secrets.psk'
ssh $TARGET 'sudo cat /run/vpn/ipsec.secrets'
```

Pass criteria:

- strongSwan: `secret = "MyStrongPassphrase123!"`
- libreswan: `: PSK "MyStrongPassphrase123!"`

5.2 Hex (0x) secret -> unquoted (binary decode)

```
ssh $TARGET sudo scli <<'EOF'
security vpn set psk secret
        0x3A5C0D50A4FA16A92813D2F662193323E22C80943E3186FF128800B1D9B478AB
security vpn save
exit
EOF
ssh $TARGET 'sudo cat /run/vpn/swanctl.secrets.psk'
ssh $TARGET 'sudo cat /run/vpn/ipsec.secrets'
```

Pass criteria:

- strongSwan: `secret = 0x3A5C...` (no quotes)
- libreswan: `: PSK 0x3A5C...` (no quotes)

5.3 Base64 (0s) secret -> unquoted

```
ssh $TARGET sudo scli <<'EOF'
security vpn set psk secret 0sQUJDREVGRw==
security vpn save
exit
EOF
ssh $TARGET 'sudo cat /run/vpn/swanctl.secrets.psk'
```

Pass criteria: `secret = 0sQUJDREVGRw==` (no quotes).

5.4 PPK ID + secret

```
ssh $TARGET sudo scli <<'EOF'
security vpn set ppk id ppk001
security vpn set ppk secret
        0x3A5C0D50A4FA16A92813D2F662193323E22C80943E3186FF128800B1D9B478AB
security vpn save
exit
EOF
ssh $TARGET 'sudo cat /run/vpn/swanctl.secrets.psk'
ssh $TARGET 'sudo cat /run/vpn/ipsec.secrets'
```

Pass criteria:

```
ppk-1 {
    id = ppk001
    secret = 0x3A5C...
}

... : PPKS "ppk001" 0x3A5C...
```

Test Group 6 — Boot-time path consistency

The boot-time `vpn-pkcs11-pin.sh` writes the same files as the SCLI `save` flow but through a different code path (shell vs Go). Verify parity.

```
ssh $TARGET 'sudo /usr/sbin/vpn-pkcs11-pin.sh && echo OK'
ssh $TARGET 'sudo cat /run/vpn/swanctl.secrets.psk'
ssh $TARGET 'sudo cat /run/vpn/ipsec.secrets'
```

Pass criteria: output is identical (same quoting rules) to the output produced by `security vpn save`.

Test Group 7 — libreswan + PSK + DN interop (NIAP open question)

NIAP NDcPP / VPN GW PP guarantees DN as a reference identifier for **certificate** authentication. For **PSK** with libreswan, the `ipsec.secrets` format documents IP / FQDN / `user@FQDN` / `%any` as indices but does not explicitly guarantee DN. This test confirms whether DN is operationally usable.

The peer (server) must be configured with the same PSK and matching identifier. Adjust the IDs and secret to match the lab server.

```
ssh $TARGET sudo scli <<'EOF'
security vpn set engine libreswan
```

```
security vpn set auth psk
security vpn set psk local-id "CN=Test Client"
security vpn set psk remote-id "CN=Test Server"
security vpn set psk secret 0x$(openssl rand -hex 32)
security vpn save
security vpn set service restart
exit
EOF
ssh $TARGET 'ipsec auto --up vpn 2>&1 | tail -20'
ssh $TARGET 'journalctl -u ipsec --since "1 minute ago" \
  | grep -iE "PSK|secret|loaded|matched|ID_KEY_ID|ID_DER_ASN1_DN"
        | head -20'
```

Pass criteria:

- pluto log shows `loaded PSK secret for ...` with the configured DN
- IKE_AUTH succeeds (e.g. `IPsec SA established` or `STATE_PARENT_I2`)
- ID type is logged as `ID_DER_ASN1_DN`

Fail action: if the peer cannot match by DN, document in AGD that PSK + DN is supported on the strongSwan engine only.

Cleanup / rollback

```
ssh $TARGET 'sudo cp /var/lib/akita-vpn/pkcs11-meta.conf.bak \
  /var/lib/akita-vpn/pkcs11-meta.conf'
ssh $TARGET 'sudo /usr/sbin/vpn-pkcs11-pin.sh'
ssh $TARGET 'sudo systemctl restart ipsec strongswan'
```

Pass / Fail tracking

Group	Test	Result
1. CLI validation	1.1 valid IDs accepted	
	1.2 invalid IDs rejected	
	1.3 canonical form (trim)	
	1.4 secret validation	
2. pkcs11-meta	2.1 single-quote applied	
	2.2 sourcing parser-safe	
	2.3 PSK -> PUBKEY clears IDs	
	2.4 PUBKEY -> PSK writes IDs	
3. strongSwan	3.2 DN quoted in swanctl files	
	3.3 strongSwan starts cleanly	
4. libreswan	4.2 DN quoted in ipsec files	
	4.3 addconn parses cleanly	

Group	Test	Result
	4.4 IP / FQDN works	
5. Secret formats	5.1 ASCII quoted	
	5.2 hex (0x) unquoted	
	5.3 base64 (0s) unquoted	
	5.4 PPK formatted	
6. Boot-time path	shell output matches SCLI save	
7. NIAP interop	libreswan + PSK + DN connects	

Tester: _____ **Date**: _____ **Build / Commit**: _____ **Result summary**: ☐ All pass ☐ N pass / M fail (notes:)

End of test procedure.